CONTINUATION-IN-PART: Universal Computation Target with Multi-Language Capability Compiler

Church-Turing Meta-Machine: Language-Independent Capability-Secured Instruction Set with Compiler-Verified Resident Object Model and Upload-Driven Abstraction Lifecycle

---

Inventor: Kenneth James Hamer-Hodges

Parent Application: Church-Turing Meta-Machine: Hardware-Enforced Lambda Calculus with the LAMBDA Instruction, NULL Capability Type, and Atomic Abstraction Architecture (Filed February 12, 2026)

Related Applications:

- Pure Church Lambda Processor: Architectural Exclusion of Turing-Domain Instructions as a Security Enforcement Mechanism (Filed February 2026)
- Church-Turing Meta-Machine: Dual-Gate Trusted Security Base with Hardware-Enforced Lambda Calculus, Deterministic Garbage Collection, and Architectural Vulnerability Elimination (Filed February 2026)

Classification: Computer Architecture; Hardware Security; Capability-Based Computing; Compiler Architecture; Multi-Language Compilation; Upload-Driven Software Lifecycle; Scale-Free Security

---

TITLE OF THE INVENTION

Language-Independent Capability-Secured Instruction Set Architecture with Multi-Language Compiler, Resident Object Model, and Upload-Driven Abstraction Lifecycle in a Capability-Based Processor

---

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation-in-part of the CTMM patent applications filed February 2026, which disclose: the Golden Token (GT) capability architecture with domain purity enforcement separating Turing-domain (R, W, X) from Church-domain (L, S, E) permissions; the LAMBDA instruction for lightweight in-scope code application; the atomic abstraction architecture; the dual-gate Trusted Security Base (mLoad + mSave); and the Pure Church variant demonstrating that Turing-domain instructions can be

architecturally excluded without loss of computational completeness.

The present application extends those disclosures by demonstrating that the capability-secured instruction set constitutes a universal computation target — a fixed, language-independent instruction set to which multiple programming paradigms (imperative, functional, declarative) can be compiled while preserving the hardware security guarantees of the parent architecture. The invention introduces: (1) a multi-language compiler with paradigm-specific front-ends and a shared capability-aware back-end; (2) a Resident Object Model that maps source-language capability references to hardware c-list offsets; (3) a single-lump abstraction model where one namespace entry describes both code and capability list, split by hardware at CALL time; (4) an upload-driven abstraction lifecycle managed by a sole namespace authority (Navana); and (5) the architectural separation of correctness (compiler domain) from security (hardware domain).

FIELD OF THE INVENTION

The present invention relates to a compilation system and abstraction lifecycle for a capability-secured processor, wherein source programs written in multiple programming languages — including but not limited to imperative languages (JavaScript, C-like subsets) and functional languages (Haskell, lambda calculus) — are compiled to a fixed set of 20 capability-secured instructions, producing self-describing upload objects that are validated and installed by a sole namespace authority through a uniform upload protocol.

BACKGROUND

The Language-Architecture Coupling Problem

All known processor instruction sets are designed for, or evolve toward, a single programming paradigm. x86 and ARM are optimized for imperative C/C++ compilation. LISP machines (Symbolics, MIT) were designed for functional LISP. The Reduceron and GRIP were designed for Haskell-style graph reduction. Java processors (picoJava) targeted Java bytecode. In every case, the instruction set is coupled to the source language — the ISA is a hardware implementation of one language's computational model.

This coupling has three consequences:

1. Security is language-dependent. Safety guarantees available in one language (Haskell's type safety, Rust's ownership) are absent when the same hardware runs a different language (C, assembly). The hardware provides no language-independent security floor.

2. Capability systems are bolted on. CHERI, for example, adds capability checks to an

existing ISA (MIPS, RISC-V, ARM). The capability mechanism is an addition to the instruction set, not a property of it. A non-capability instruction can still execute and bypass the protection model unless the compiler cooperates.

3. Compilation is trusted. The compiler is inside the trusted computing base — a compiler bug can produce code that violates the security model. This is because the hardware relies on the compiler to emit correct instructions, and the instruction set does not architecturally prevent misuse.

The Discovery: Language-Independent Capability Target

The parent CTMM application discloses 20 instructions divided into two domains: 10 Church-domain (LOAD, SAVE, CALL, RETURN, CHANGE, SWITCH, TPERM, LAMBDA, ELOADCALL, XLOADLAMBDA) and 10 Turing-domain (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR), plus a shared RETURN. Domain purity is enforced by hardware: a GT cannot hold permissions from both domains simultaneously.

The present invention recognizes that this 20-instruction set is not merely "sufficient" for computation — it is a universal computation target in the following precise sense: source programs from fundamentally different programming paradigms (imperative and functional) compile to the same instruction set, producing semantically equivalent machine code, while the hardware security model (capability validation, domain purity, bounds checking) applies identically regardless of source language.

This has been demonstrated by reduction to practice: the CLOOMC++ compiler compiles both JavaScript (imperative, C-like syntax, explicit control flow) and Haskell (functional, lambda calculus, pattern matching) to the same 20 instructions, producing identical upload objects processed by the same namespace authority. The instruction IADD DR4, DR0, #1 is emitted for both result = who + 1 (JavaScript) and successor(n) = n + 1 (Haskell). The hardware cannot distinguish the source language — and does not need to.

The Compiler-Security Separation

A critical insight of the present invention is the architectural separation of correctness from security:

- Correctness is the compiler's responsibility. A compiler bug produces wrong answers — incorrect register allocation, wrong branch targets, miscomputed values. These are functional defects.
- Security is the hardware's responsibility. The capability model constrains from below. A compiler bug cannot: forge a Golden Token; escape the lump bounds enforced by the namespace entry; access a capability not present in the c-list; cross domain boundaries (Church ↔ Turing); or bypass the mLoad/mSave validation pipeline.

This separation means the compiler is outside the trusted computing base. No compiler bug, in any source language, can produce a security violation. The hardware enforces the invariants regardless of what code the compiler generates.

DETAILED DESCRIPTION

The 20-Instruction Universal Target

The Church Machine instruction set comprises 20 instructions in two domains:

Domain	Instruction	Encoding	Function
Church	LOAD	00000	Load GT from c-list (CR6) into capability register; req
Church	SAVE	00001	Save GT from capability register to c-list; requires S p
Church	CALL	00010	Enter abstraction scope via E-GT; splits lump into CR
Church	RETURN	00011	Exit abstraction scope; restore caller context
Church	CHANGE	00100	Modify GT permissions (reduce only)
Church	SWITCH	00101	Context switch between threads
Church	TPERM	00110	Test GT permissions; FAULT on failure
Church	LAMBDA	00111	Apply function in-scope via X permission
Church	ELOADCALL	01000	Fused LOAD+CALL (optimization)
Church	XLOADLAMBDA	01001	Fused LOAD+LAMBDA (optimization)
Turing	DREAD	01010	Read data word from DATA object via R permission
Turing	DWRITE	01011	Write data word to DATA object via W permission
Turing	BFEXT	01100	Extract bitfield from data register
Turing	BFINS	01101	Insert bitfield into data register
Turing	MCMP	01110	Compare two data registers; set condition flags
Turing	IADD	01111	Integer addition
Turing	ISUB	10000	Integer subtraction
Turing	BRANCH	10001	Conditional branch (ARM-style condition codes)
Turing	SHL	10010	Shift left
Turing	SHR	10011	Shift right

All instructions share a uniform 32-bit encoding: opcode[5] | cond[4] | dst[4] | src[4] | imm[15].

ARM-style condition codes (EQ, NE, CS, CC, MI, PL, VS, VC, HI, LS, GE, LT, GT, LE, AL, NV) apply to every instruction, providing conditional execution without separate branch-and-execute sequences.

The Multi-Language Compiler (CLOOMC++)

The CLOOMC++ compiler is a multi-front-end, single-back-end compiler targeting the 20-instruction Church Machine ISA. Each front-end parses a different source language; all front-ends share the same Resident Object Model and code generator.

Front-End Architecture

Language Detection: The compiler auto-detects the source language by examining syntactic markers. The = sign after method parameter lists, -- comment markers, and if...then...else syntax indicate Haskell. Curly braces, var declarations, and // comments

indicate JavaScript. This detection is performed before parsing begins.

JavaScript Front-End (imperative paradigm):

The JavaScript front-end parses a subset of JavaScript sufficient for system programming:

Source Construct	**Church Machine Instruction(s)**
var x = expr	Register allocation + expression code
x = expr	Expression evaluation → target register
x + y	IADD DRz, DRx, DRy
x - y	ISUB DRz, DRx, DRy
x << n	SHL DRz, DRx, #n
x >> n	SHR DRz, DRx, #n
if (x == y) {...}	MCMP DRx, DRy + BRANCH.EQ
while (cond) {...}	Label + MCMP + BRANCH (loop)
call(Abs.Method(args))	LOAD CR from c-list + CALL
read(cr, offset)	DREAD DRx, CRy, #offset
write(cr, offset, val)	DWRITE DRx, CRy, #offset
bfext(val, pos, width)	BFEXT DRz, DRx, #pos, #width
bfins(dst, val, pos, width)	BFINS DRz, DRx, #pos, #width
return(val)	Move result to DR1 + RETURN

Haskell Front-End (functional paradigm):

The Haskell front-end parses a subset of Haskell supporting lambda calculus, pattern matching, and algebraic data:

Source Construct	**Church Machine Instruction(s)**
\x -> body	LAMBDA (Church function application)
f x	XLOADLAMBDA or CALL (function application)
let x = expr in body	Register binding + expression + body code
case x of { 0 -> a, 1 -> b, _ -> c }	MCMP DRx, #0 + BRANCH.EQ + MCMP DRx, #1 + BRANCH.EQ + def...
if p then a else b	MCMP DRp, #0 + BRANCH.EQ (to else) + then-code + BRANCH (...
x + y	IADD DRz, DRx, DRy
x * y	Iterative IADD loop (MCMP + BRANCH + IADD)
(a, b)	SHL DRa, #16 + BFINS DRb (pair encoding)
fst p	SHR DRp, #16 (first element extraction)
snd p	BFEXT DRp, #0, #16 (second element extraction)
succ n	IADD DRn, DRn, #1
pred n	ISUB DRn, DRn, #1

Critical observation: Both front-ends produce the same instruction for equivalent operations. result = who + 1 (JavaScript) and successor(n) = n + 1 (Haskell) both emit IADD DR4, DR0, #1. The hardware cannot distinguish the source language. This is the defining property of a universal computation target.

The Resident Object Model

The Resident Object Model (ROM) is the compiler's representation of the abstraction's capability environment. It serves as the bridge between source-language names and hardware c-list offsets.

Key insight: The c-list IS the compiler's symbol table for external references. When source code references an external abstraction (e.g., call(Memory.Allocate(size)) in JavaScript or Memory.allocate size in Haskell), the compiler must resolve this to a LOAD instruction targeting a specific c-list slot in CR6. The ROM maps:

```
Source name "Memory"  →  c-list offset 0  →  LOAD CR_temp, CR6, #0
Source name "Mint"    →  c-list offset 1  →  LOAD CR_temp, CR6, #1
```

The ROM is generated directly from the upload's capabilities array — the same source of truth that the namespace authority (Navana) uses to populate the physical c-list at installation time. The compiler never guesses offsets; it reads them from the capability declaration.

Risk mitigation (R005): If the ROM were generated independently of the capabilities array, a mismatch could cause the code to LOAD the wrong GT and CALL the wrong abstraction — capability confusion. By deriving both from the same source, this class of error is eliminated by construction.

Calling Convention

The compiler enforces a fixed register convention across all source languages:

Registers	Role	Saved By
DR0	Hardwired zero	—
DR1-DR3	Arguments and return values	Caller
DR4-DR11	Local variables	Callee
DR12-DR15	Temporaries (compiler scratch)	Caller

This convention is architectural, not advisory. The compiler allocates registers within these ranges regardless of source language. A JavaScript method and a Haskell method use the same register protocol, enabling cross-language CALL sequences — a JavaScript abstraction can CALL a Haskell-compiled abstraction through the standard E-GT mechanism with no adaptation layer.

The Single-Lump Abstraction Model

The present invention introduces a single-lump model for abstractions: one namespace (NS) entry describes one contiguous memory allocation (the "lump") that contains both code and capability list. The CALL instruction splits the lump into two domain-pure regions at runtime.

NS Entry Word1 Layout

```
Bit 31:    B (Bind) — can this GT be saved to another c-list?
Bit 30:    F (Far) — is this a remote/network resource?
Bit 29:    G (GC) — garbage collection liveness flag
```

```
Bit 28:    Chain — reserved for linked structures
Bits 27:26: Type — 00=NULL, 01=Inform, 10=Outform, 11=Abstract
Bits 25:17: clistCount — number of c-list entries (0-511)
Bits 16:0:  Limit — size of allocation (17 bits, max 131072 words)
```

clistCount is the architectural key. When clistCount > 0, the CALL instruction knows this is an abstraction with a capability list, and splits accordingly. When clistCount = 0, the NS entry describes a plain data object (no c-list, no code/data split).

CALL Lump Split

When the CALL instruction processes an Inform E-GT with clistCount > 0:

1. mLoad validation: GT type check → version match → seal verify → bounds check → E-permission check → F-bit check → deliver NS entry
2. Parse word1: Extract clistCount and limit from the NS entry
3. Compute split point: clistStart = (limit + 1) - clistCount
4. Create CR14 (Code Region): location = base, limit = clistStart - 1, permissions = X-only (hardcoded)
5. Create CR6 (C-List Region): location = base + clistStart, limit = clistCount - 1, permissions = L-only (hardcoded)
6. Set PC = 0: Execution begins at the first instruction of the code region

Critical security invariant (R001): CR14 permissions are hardcoded to X-only (execute). CR6 permissions are hardcoded to L-only (load capability). These permissions are not derived from the E-GT or the NS entry — they are architectural constants enforced by the CALL instruction. This prevents:

- Code reading its own capabilities as data (CR14 cannot Load)
- C-list entries being executed as code (CR6 cannot eXecute)
- Any cross-domain contamination between code and capability regions

Lump Memory Layout

```
┌──────────────────────────────────────┐ offset 0
│ Method table + compiled code         │ → CR14 (Turing domain, X-only)
│ (produced by [CLOOMC](https://sipantic.blogspot.com/2025/03/xx.html)++ compiler)      |
├──────────────────────────────────────┤ codeEnd
│ FREESPACE (power-of-2 padding)       │ inaccessible (beyond code limit)
├──────────────────────────────────────┤ clistStart = allocSize - clistCount
│ C-list (Golden Token slots)          │ → CR6 (Church domain, L-only)
│ (populated by Navana from upload)    │
└──────────────────────────────────────┘ allocSize (power-of-2)
```

The freespace region is architecturally inaccessible — it lies beyond CR14's limit and before CR6's base. This provides growth room for code updates without reallocating the lump.

The Upload-Driven Abstraction Lifecycle

The present invention introduces a uniform upload protocol for creating, updating, and

removing abstractions. Every abstraction — whether compiled from JavaScript, Haskell, or hand-assembled — enters the system through the same upload format:

```
{
    "abstraction": "Name",
    "type": "abstraction",
    "grants": ["E"],
    "capabilities": [
        { "target": 7, "name": "Memory", "grants": ["E"] },
        { "target": 6, "name": "Mint", "grants": ["E"] }
    ],
    "methods": [
        { "name": "MethodName", "code": [0x12345678, 0x9ABCDEF0] }
    ]
}
```

Fields:

- abstraction: Problem-oriented name (human-readable)
- type: "abstraction" (code + c-list) or "namespace" (isolated child namespace)
- grants: Permissions granted to callers via the E-GT
- capabilities: C-list wiring — each entry specifies a target NS index, name, and delegated permissions
- methods: Compiled code — each method is a named sequence of 32-bit instruction words

The upload is both the IDE's output format and the runtime's input format. The compiler produces uploads; the namespace authority (Navana) consumes them.

Navana: Sole Namespace Authority

Navana is the sole writer of namespace table entries after boot. This is an architectural invariant:

1. Boot (mElevation): A single raw write installs Navana's own NS entry — the one exception to the rule
2. All subsequent NS writes: Routed through Navana.Add, which finds a free slot, writes the 3-word NS entry (location, word1 with clistCount and type, seal), and returns the NS index + version
3. Abstraction creation: Navana.Abstraction.Add processes the upload: allocates a power-of-2 lump via Memory, writes compiled code to the code region, populates the c-list with delegated GTs, creates the NS entry via Navana.Add, and forges the E-GT
4. Drop mElevation: After boot, machine-level write access is permanently relinquished — Navana operates through its own capabilities thereafter

Upload validation (R007): Navana validates every upload before installation:

- codeSize + clistCount <= allocatedSize (no overlap between code and c-list)
- Each capability target exists AND the creator holds sufficient permissions to delegate
- clistCount <= 511 (fits in 9 bits of word1)

- allocatedSize is a valid power-of-2
- Integer overflow/underflow checks on all size arithmetic

Mint delegates to Navana: The Mint abstraction (GT lifecycle management) delegates NS entry creation to Navana.Add. Mint.Create performs domain validation and GT forging; the actual namespace write is Navana's exclusive authority.

GT Type System

The architecture defines four GT types enforced by the hardware type field (bits 1:0):

Type	Encoding	Semantics
NULL	00	Zero value, no capability. Any dereference → FAULT
Inform	01	GT points to memory via an NS entry. Used for all abstrac...
Outform	10	GT points to remote memory. F-bit set automatically. Cros...
Abstract	11	GT IS the value. No memory pointer. Constants (pi), immut...

All abstractions use Inform (01) GTs. The clistCount field in word1 distinguishes abstractions (clistCount > 0) from plain data objects (clistCount = 0).

Scale-Free Security

The abstraction model is scale-free — the same security mechanism (GT + NS entry + lump + CALL split) applies identically at every scale:

- Single child: One namespace, one set of abstractions, one parent's c-list = parental approval
- Family: Multiple sibling namespaces, each isolated, parent holds E-GTs to grant/revoke access
- School: Teacher namespace + student namespaces, Negotiate abstraction for dual-approval grants
- Organization: Hierarchical namespaces, Outform GTs for cross-boundary access, Tunnel for networking

The upload protocol, the namespace authority model, and the capability validation pipeline are identical at every scale. No "enterprise mode" or "admin privilege" exists — the same 20 instructions, the same mLoad pipeline, the same CALL split.

REDUCTION TO PRACTICE

Multi-Language Compilation Proof

The CLOOMC++ compiler has been implemented as a working system (simulator/cloomc_compiler.js) with both JavaScript and Haskell front-ends. The following

abstractions have been compiled and executed:

JavaScript front-end — 7 resident objects:

1. Hello: Single method (Greet) — IADD + RETURN (3 instructions)
2. Counter: Two methods (Increment, Add) — IADD + RETURN each
3. Memory: Two methods (Allocate, Free) — DREAD + SHR + SHL + DWRITE + IADD + RETURN
4. Mint: Two methods (Create, Revoke) — LOAD + CALL + RETURN (c-list wiring via ROM)
5. Navana: Nine methods — Init, Add, Remove, Abstraction.Add/Remove/Update, Manage, Monitor, IDS
6. Salvation: Four methods — LOAD, TPERM, LAMBDA, TransitionToNavana (boot verification)
7. Scheduler/Stack/DijkstraFlag: Stub methods (RETURN-only, awaiting Phase 2 compilation)

Haskell front-end — 4 resident objects:

1. ChurchMath: Five methods — successor (IADD), add (IADD), multiply (IADD loop with MCMP+BRANCH), predecessor (MCMP+BRANCH+ISUB), isZero (MCMP+BRANCH)
2. ChurchPair: Four methods — makePair (SHL+BFINS), first (SHR), second (BFEXT), swap (SHR+BFEXT+SHL+BFINS)
3. ChurchCase: Three methods — factorial (MCMP+BRANCH+ISUB), classify (MCMP chain), abs (MCMP+BRANCH+ISUB)
4. ChurchLambda: Four methods — identity (RETURN), constant (RETURN), double_succ (IADD+IADD), letExample (IADD+IADD)

Universal target proof: The instruction IADD DR4, DR0, #1 appears in both Hello.Greet (JavaScript: result = who + 1) and ChurchMath.successor (Haskell: method successor(n) = n + 1). The hardware executes identical machine code for equivalent operations across paradigms.

Boot Sequence Proof

The boot sequence processes an upload array of 11 abstractions through the following stages:

1. FAULT_RST: Zero all registers
2. mElevation: Raw write Navana's NS entry (sole exception to Navana-only writes)
3. Process remaining uploads through Navana.Add
4. Drop mElevation permanently
5. CALL Salvation → Salvation validates security pipeline → Transition to Navana
6. Navana runs forever (no RETURN)

All Phase 1 abstractions (Memory, Mint, Navana) run real CLOOMC++-compiled code.

Phase 2-4 abstractions (Scheduler, Stack, DijkstraFlag, hardware attachments, math, lambda, social, IDE, internet, GC) have stub methods (RETURN-only) until compiled.

Security Risk Register

Nine risks (R001-R009) have been identified, documented, and resolved:

Risk	Severity	Description	Status
R001	CRITICAL	CALL must hardcode CR14=X, CR6=1	RESOLVED
R002	MEDIUM	25-bit FNV seal strength	ACCEPTED (adequate for target hardware)
R003	LOW	Boot raw write single point of failure	RESOLVED
R004	HIGH/LOW	Compiler incorrect code generation	RESOLVED (security unaffected by compiler bugs)
R005	MEDIUM	C-list offset mismatch	RESOLVED (ROM derived from capabilities array)
R006	MEDIUM	Haskell closure variable capture	RESOLVED (closures capture DR only, never CR)
R007	HIGH	Upload validation integer underflow	RESOLVED (Navana validates all uploads)
R008	MEDIUM	Register spilling / calling convention	RESOLVED (fixed convention DR0-3/DR4-11/DR12-15
R009	FOUNDATIONAL	Namespace isolation guarantee	SECURE (all contingencies resolved)

End-to-End Verification

Automated end-to-end testing confirms:

- JavaScript source compiles and produces [JavaScript] tagged output with correct hex instruction words
- Haskell source compiles and produces [Haskell] tagged output with correct hex instruction words
- Both produce valid compiled abstractions processed by Navana.Abstraction.Add
- Boot sequence completes with all 11 abstractions installed
- CR14 and CR6 are correctly set from single NS entry with clistCount split

---

PROPOSED CLAIMS

Claim 24 — Universal Computation Target Instruction Set

A processor instruction set architecture comprising:

(a) a fixed set of instructions divided into two domains — a Church domain for capability operations (LOAD, SAVE, CALL, RETURN, CHANGE, SWITCH, TPERM, LAMBDA, ELOADCALL, XLOADLAMBDA) and a Turing domain for data operations (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR);

(b) wherein all instructions share a uniform encoding format comprising an opcode field, a condition code field, a destination register field, a source register field, and an immediate value field;

(c) wherein the instruction set constitutes a universal computation target, demonstrated by the successful compilation of source programs from at least two fundamentally different programming paradigms — imperative and functional — to the same instruction set, producing semantically equivalent machine code for equivalent operations;

(d) wherein the hardware security model — comprising capability token validation, domain purity enforcement, bounds checking, and permission verification through a trusted gate pipeline — applies identically to all compiled code regardless of source language;

(e) wherein the compiler that translates source programs to the instruction set is architecturally outside the trusted computing base, such that no compiler bug in any source language can produce a security violation, because the hardware enforces capability invariants independent of the code generated.

Claim 25 — Multi-Language Capability Compiler with Resident Object Model

A compilation system for the processor of Claim 24, comprising:

(a) a multi-front-end compiler architecture wherein each front-end parses a different source language and all front-ends share a common back-end that generates instructions from the fixed instruction set of Claim 24;

(b) a language detection mechanism that automatically identifies the source language from syntactic markers before parsing begins;

(c) a Resident Object Model that maps source-language references to external abstractions (capability names) to hardware c-list offsets, wherein the mapping is derived directly from the upload's capability declaration — the same source of truth used by the namespace authority to populate the physical c-list at installation time;

(d) a fixed calling convention enforced across all source languages, wherein a first set of data registers is designated for argument passing and return values, a second set for callee-saved local variables, and a third set for caller-saved temporaries;

(e) wherein the Resident Object Model ensures that a capability reference in source code (e.g., call(Memory.Allocate(size)) in an imperative language or Memory.allocate size in a functional language) compiles to the same hardware instruction sequence — a LOAD from the correct c-list offset followed by a CALL — regardless of source language;

(f) wherein the compiler output is a self-describing upload object containing the abstraction name, capability declarations, and compiled method code, suitable for processing by the namespace authority of Claim 28.

Claim 26 — Single-Lump Abstraction Model with Hardware CALL Split

A method of managing abstractions in the processor of Claim 24, comprising:

(a) allocating a single contiguous memory region (lump) for each abstraction, wherein the lump contains both compiled code and a capability list (c-list);

(b) describing the lump with a single namespace entry, wherein the namespace entry's word1 field contains a clistCount value indicating the number of c-list entries;

(c) upon execution of a CALL instruction targeting the abstraction:

- computing a split point as clistStart = (limit + 1) - clistCount;
- creating a first context register (CR14) pointing to the code region with permissions hardcoded to execute-only (X);
- creating a second context register (CR6) pointing to the c-list region with permissions hardcoded to load-only (L);
- setting the program counter to zero;

(d) wherein the code region permissions (X-only) and c-list region permissions (L-only) are architectural constants enforced by the CALL instruction, not derived from the Golden Token or namespace entry permissions;

(e) wherein this hardcoded domain split prevents code from reading capabilities as data and prevents c-list entries from being executed as code, maintaining domain purity across the code/capability boundary.

Claim 27 — Capability-Type Taxonomy with Architectural Semantics

The processor of Claim 24, wherein each Golden Token contains a two-bit type field with the following architectural semantics:

(a) NULL (00): zero value, no capability; any dereference produces an immediate FAULT;

(b) Inform (01): the GT points to a memory-backed object described by a namespace entry; used for all abstractions and data objects; the clistCount field in the namespace entry distinguishes abstractions (clistCount > 0) from plain data objects (clistCount = 0);

(c) Outform (10): the GT points to a resource in a remote namespace; the Far (F) bit is set automatically; used for cross-namespace and network-transparent access;

(d) Abstract (11): the GT IS the value; no memory pointer or namespace entry is required; used for mathematical constants, immutable credentials, and escale variables;

wherein the type field is validated by the hardware trusted gate pipeline on every access, and type-specific behavior (memory dereference for Inform, network routing for Outform, immediate value for Abstract) is performed by the hardware without software intervention.

Claim 28 — Sole Namespace Authority with Upload-Driven Lifecycle

A method of managing the namespace table in the processor of Claim 24, comprising:

(a) designating a single abstraction (Navana) as the sole authority permitted to write

namespace table entries after the boot sequence;

(b) during boot, performing exactly one privileged write (mElevation) to install Navana's own namespace entry, and thereafter permanently relinquishing privileged write access;

(c) routing all subsequent namespace table writes through Navana.Add, which finds a free namespace slot, writes the three-word namespace entry (location, word1 with clistCount and type and seal), and returns the namespace index and version;

(d) processing abstraction creation through Navana.Abstraction.Add, which:

- validates the upload object (code size + c-list count within allocation bounds, capability delegation authority verified, integer overflow checks);
- allocates a power-of-2 memory lump via the Memory abstraction;
- writes compiled code to the code region of the lump;
- populates the c-list region with delegated Golden Tokens;
- creates the namespace entry via Navana.Add;
- forges an Inform E-GT (Enter permission, type=01) and returns it to the creator;

(e) wherein other abstractions (including Mint, the GT lifecycle manager) delegate namespace entry creation to Navana, ensuring a single point of namespace authority and a single validation path for all abstractions regardless of source language or creation method.

Claim 29 — Compiler-Security Architectural Separation

The compilation system of Claim 25, wherein:

(a) the compiler is architecturally outside the trusted computing base of the processor;

(b) a compiler bug can produce functionally incorrect code (wrong register allocation, incorrect branch targets, miscomputed values) but cannot produce a security violation;

(c) security invariants enforced by the hardware independent of compiler output include:

- Golden Token validation (version match, seal verification, permission check) on every memory access;
- domain purity (Church-domain and Turing-domain permissions cannot coexist on a single GT);
- bounds checking (code cannot access memory beyond the lump limits set by the namespace entry);
- CALL split hardcoding (CR14=X-only, CR6=L-only regardless of compiled code);
- c-list authority (code can only LOAD GTs present in its c-list; no GT can be forged by any instruction sequence);

(d) wherein this separation holds for all source languages processed by the compiler, such that adding a new language front-end to the compiler cannot weaken the security model — the hardware provides a language-independent security floor.

Claim 30 — Cross-Paradigm Capability Interoperability

The compilation system of Claim 25, wherein:

(a) abstractions compiled from different source languages can invoke each other through the standard CALL mechanism, using the same E-GT protocol, register convention, and c-list wiring;

(b) a first abstraction compiled from an imperative language and a second abstraction compiled from a functional language can hold E-GTs to each other in their respective c-lists;

(c) the CALL instruction, the mLoad validation pipeline, and the lump split mechanism operate identically regardless of the source languages of the caller and callee;

(d) no adaptation layer, foreign function interface, or runtime type conversion is required for cross-paradigm invocation — the hardware protocol is the sole interface.

Claim 31 — Scale-Free Security Architecture

The processor of Claim 24, wherein:

(a) the abstraction model (GT + namespace entry + lump + CALL split) applies identically at every organizational scale — individual user, family, school, enterprise, and inter-organizational;

(b) namespace isolation is achieved by each entity having its own namespace table, memory region, and set of Golden Tokens;

(c) access across namespace boundaries requires a valid Outform GT (type=10) with the Far bit set, processed through the same mLoad validation pipeline;

(d) capability delegation across scales uses the same upload protocol and Navana validation path;

(e) revocation at any scale is achieved by incrementing the version number in the namespace entry, instantly invalidating all outstanding GT copies — with no "revocation list" propagation delay;

(f) no "admin mode," "superuser privilege," "root access," or "enterprise tier" exists — the same 20 instructions, the same mLoad pipeline, and the same CALL split apply to every user at every scale.

PRIOR ART DISTINCTION

System	Multi-Language Target	Capability Security	Compiler Outside TCB	Upload Lifecycle	Scale-Free
JVM (Sun/Oracle)	Yes (Java, Kotlin, Scala)	No	No	No	No
CLR (.NET/Microsoft)	Yes (C#, F#, VB)	No	No	No	No

LLVM	Yes (C, Rust, Swift)	No	No	No	No
CHERI (Cambridge)	No (C/C++ focus)	Yes	No	No	No
WebAssembly (W3C)	Yes (C, Rust, Go)	No (sandbox only)	No	No	No
LISP Machines	No (LISP only)	No	No	No	No
Reduceron	No (Haskell only)	No	No	No	No
CTMM (parent)	No (single language)	Yes	Partially	No	Partially
Church Machine (this CTP)	Yes	Yes	Yes	Yes	Yes

Key distinctions from JVM/CLR/LLVM: These are multi-language compilation targets, but they lack hardware capability security. The compiler is inside the trusted computing base — a compiler bug can produce code that violates the security model (buffer overflows in JVM native methods, unsafe blocks in .NET, undefined behavior in LLVM-compiled C).

Key distinction from CHERI: CHERI adds capabilities to an existing ISA but is designed primarily for C/C++ memory safety. The compiler must cooperate with the capability model. The Church Machine's ISA is inherently capability-secured — every instruction operates through GTs — making the compiler's cooperation unnecessary for security.

Key distinction from WebAssembly: Wasm is a multi-language target with a sandbox model, but sandboxing is weaker than capability security — a Wasm module has ambient authority within its sandbox. The Church Machine's c-list model provides fine-grained authority: each abstraction can only access the specific capabilities delegated to it.

FIGURES (Proposed)

Figure 24: Multi-Language Compilation to Universal Target

Diagram showing JavaScript and Haskell source code flowing through language-specific front-ends into the shared Resident Object Model and code generator, producing identical instruction words for equivalent operations, with the compiled output feeding into Navana.Abstraction.Add.

Figure 25: Single-Lump CALL Split

Diagram showing a single namespace entry pointing to a contiguous lump, with the CALL instruction splitting the lump at the clistStart boundary into CR14 (X-only code) and CR6 (L-only c-list), with freespace between.

Figure 26: Compiler-Security Separation

Diagram showing the trust boundary: compiler produces code words (correctness domain, outside TCB); hardware validates GTs, enforces bounds, hardcodes domain permissions (security domain, inside TCB). Arrows showing that compiler bugs affect correctness within the security perimeter but cannot breach it.

Figure 27: Upload-Driven Lifecycle

Diagram showing the lifecycle: source code → CLOOMC++ compiler → compiled abstraction → Navana.Abstraction.Add (validation) → Memory.Allocate (power-of-2 lump) → code write + c-list populate → NS entry creation → E-GT forge → return to creator.

Figure 28: Scale-Free Security Model

Diagram showing identical security mechanisms (GT + NS entry + CALL split + mLoad) applied at individual, family, school, and organizational scales, with Outform GTs bridging namespace boundaries.

ABSTRACT

A capability-secured processor instruction set architecture that serves as a universal computation target for multiple programming paradigms. The fixed 20-instruction set — comprising 10 Church-domain capability operations and 10 Turing-domain data operations — accepts compiled output from imperative languages (JavaScript) and functional languages (Haskell) through a multi-front-end compiler with a shared Resident Object Model and code generator. The compiler maps source-language capability references to hardware c-list offsets, producing self-describing upload objects processed by a sole namespace authority (Navana) through a uniform validation and installation protocol. A single-lump abstraction model stores both code and capability list in one memory allocation, described by one namespace entry; the CALL instruction splits the lump into domain-pure regions (code=execute-only, c-list=load-only) using a clistCount field in the namespace entry. The compiler is architecturally outside the trusted computing base: no compiler bug in any source language can produce a security violation, because the hardware enforces capability invariants — token validation, domain purity, bounds checking, and permission verification — independent of the generated code. The architecture is scale-free: the same security mechanisms apply identically from individual users to organizations, with no privileged modes or administrative overrides.